Received: 16 September 2022 | Revised: 19 October 2022 | Accepted: 11 November 2022

DOI: 10.1002/eng2.12601

RESEARCH ARTICLE

Engineering Reports Open Access WILEY

Parallelizing the cryo-EM structure determination in THUNDER using GPU cluster

Zhao Wang[1] | **Huabin Ruan**[2] | **Guangwen Yang**[1] | **Xueming Li**[2]

[1]Department of Computer Science and Technology, Tsinghua University, Beijing, China

[2]School of Life Sciences, Tsinghua University, Beijing, China

Correspondence
Zhao Wang, Department of Computer Science and Technology, Tsinghua University, Beijing, 100084, China.
Email: wangzhao16@mails.tsinghua.edu.cn.

Abstract

Electron cryo-microscopy (cryo-EM) is a powerful tool utilized by biologists for understanding the mysteries of life. However, obtaining high-resolution 3D reconstructions from innumerable noisy images of macromolecules is an extremely complicated task, involving massive image analysis and calculation on a computing cluster. Although extensive efforts have been made for improving the computational efficiency, methods for completely utilizing the computing resources are still challenging for modern cryo-EM programs. Here, we designed a new computing approach specialized for GPU to optimize and maximize the computing power of a single GPU, multiple GPU, and the GPU cluster, highlighted by a well-designed cache structure and mixed computing precision of single-precision and double-precision. Our approaches achieved remarkable improvement in performance and linear scalability. At an identical cost of the hardware, three-fold more speed-up was achieved. The average parallel efficiency can increase up to 84% when multiple GPU configurations are parallelized.

KEYWORDS

3D reconstruction, cryo-EM, GPU optimization

1 | INTRODUCTION

Proteins are one of the most important components of living beings. Determination of protein structure is critical for understanding biological principles and for rational drug design. Therefore, methods of reconstructing the high-resolution 3D structure of protein molecules are important for scientists, especially structural biologists. Currently, the main approach for obtaining the 3D structure of the a protein molecule involves a combination of cryo-EM[1] and computer image processing algorithms.[2] Meaning, a large number of 2D images with different rotation angles are collected from cryo-EM devices,[3] followed by analysis and processing of the 2D image data using related image processing algorithms,[4] which finally form the 3D structure of the corresponding protein molecule.

Nowadays, certain well-known software systems can be used to analyze and process cryo-EM data, such as RELION,[5] cryoSPARC,[6] and THUNDER.[7] The basic principles inside these published software systems are the gridding and

Zhao Wang and Huabin Ruan contributed equally to this work.

gradient-descent algorithms, or their combinations.[8] Generally speaking, the gridding algorithm is a robust method but with limited accuracy and high computing costs. THUNDER proposed a new particle-filter algorithm based on the previous Bayesian-likelihood optimization algorithm, which has been used to obtain optimal Bayesian estimations for nonlinear/non-Gaussian tracking problems.[9,10]

Furthermore, the software systems mentioned above had problems in terms of performance. RELION and THUNDER both have a CPU version based on MPI. The common problem of the CPU version is that the limited number of CPU cores leads to low parallelism and throughput. Therefore, the computation performance of CPU version is still not satisfactory when dealing with large-scale datasets. Compared to CPU, GPU can effectively deal with computationally intensive applications. RELION and cryoSPARC both have a GPU version. However, both RELION's GPU[11] and CPU versions show poor scalability. Owing to the inability to achieve multi-GPU collaborative computing, its performance improvement space is extremely limited. Therefore, development of high-performance applications for processing cryo-EM datasets is still a big challenge.

Considering that THUNDER has a robust algorithm and excellent scalability in performance; we designed and implemented a GPU version of THUNDER with high throughput. To take complete advantage of GPU's specific high concurrency computing capability, we redesigned and implemented the original 3D reconstruction algorithm of THUNDER from the following three aspects:

1. We adjusted and optimized THUNDER's calculation logic to circumvent the problem of low GPU memory capacity, which is unable to handle large-sized particle images and large-scale datasets.
2. On the premise of guaranteeing the correctness of the calculation results, we used floating-point numbers with different precision in different parts of the algorithm to maximize throughput and reduce storage consumption.
3. To take advantage of high-speed GPU on-chip memory, we carefully designed data storage strategies for data with different accessing pattern.

We have implemented a high-performance GPU version of the THUNDER algorithm called gTHUNDER via design and transplantation of THUNDER in the three aspects mentioned before. A large number of experiments with large-scale datasets show that under the same computing hardware cost, gTHUNDER is more than three times faster than the original well-optimized CPU version. Meanwhile, experimental results show that gTHUNDER has good linear scalability.

2 | MATERIALS AND METHODS

Most of the current 3D reconstruction algorithms[12,13] in cryo-EM use Bayesian-likelihood approaches, which are implemented on EM (expectation-maximization) algorithm by iterative optimization of a statistical model.[14,15] Each iteration begins with a guess of the particle structure, then aligns the cryo-EM images with the structure, and averages the aligned images to update the structure.[16] The iterative process consists of three phases, namely global search, local search, and contrast transfer function (CTF) search. The initial iterations of the iterative algorithm are very inaccurate for guessing the structure, so the structure parameters should be searched globally in the parameter space. After several iterations, the global search becomes unable to increase the structure resolution, so the local search of the parameters is required. After the local search cannot significantly improve the resolution; we need to correct the CTF to further optimize the structure resolution, so the CTF search is required.

Figure 1 shows a simplified core algorithm flow of these Bayesian-likelihood approaches. There are two fundamental steps called expectation and maximization in one iteration.[17,18] In the expectation step, or E-step, our primary goal is to find the best alignment parameters such as rotation angles and translations based on the input image. The way to find the best alignment parameters is to project the 3D reference model according to the sampling rotation angles and translations, then calculate the weights by the difference between the input images and the projected images, and choose the largest weights. Then, in the maximization step, or M-step, alignment parameters found in the E-step will be used to average the particle images to update the 3D model. In light of these algorithm characteristics; we propose using three aspects of GPU to achieve more speed-up than previous CPU programs based on THUNDER.

ALGORITHM 1. Framework of parameter search in the E-step for CPU

Input: Rotation angles (R), Translations (T), Images (I), Volume (V).
Output: The probability density weight vector of R and T.

Initializing R and T
for $i \leftarrow 0$ to $NT - 1$ **do**
    extract in-plane translation T_i from T
    use T_i to calculate the translation distance called $TRAP_i$
end for
for $r \leftarrow 0$ to $NR - 1$ **do**
    extract out of plane rotation R_r from R
    calculate projection called $ROTP_r$ through R_r and V
end for
for $r \leftarrow 0$ to $NR - 1$ **do**
    for $t \leftarrow 0$ to $NT - 1$ **do**
        for $l \leftarrow 0$ to $NI - 1$ **do**
            calculate the difference $DIFF_{r,t,l}$ through I_l, $ROTPr$ and $TRAPt$
            use $DIFF_{r,t,l}$ to update the orientation weight W_r and W_t
        end for
resample R and T
    end for
end for

2.1 | Algorithm adjustment

In CPU applications, CPU computing nodes are usually able to configure hundreds of GB or even TB of memory. Owing to the limitations of chip size and heat dissipation capacity, GPUs are usually unable to configure large memory size. Therefore, for memory-intensive applications, CPU computing logic cannot be directly mapped according to GPU computing logic, because direct mapping will lead to incorrect computing results due to insufficient GPU memory capacity. In addition, compared to CPU, GPU is a multi-core processor.[19] Thus, the parallelism in CPU application is not sufficient for GPU.[20]

In summary, when mapping a memory-intensive application into GPU architecture, it is usually necessary to adjust the original CPU algorithm. The common method is to split the input data into small data blocks, adjust the processing logic of the CPU algorithm such that it can pipeline the small data blocks in sequence in an iterative manner. Ideally, the communication in sequence should follow a data management that minimizes data transfers between CPU and GPU, which would make the total time consumed by a whole sequence close to the time consumed by the sum of its calculation functions.[21]

Figure 1 shows that in the E-step, the best orientation parameters, including rotation angles and translations, are searched by comparing input images and projection images. In previous algorithms, all the rotations are traversed first, followed by translations and images. This means that for any given rotation and translation in the set, the differences between all the input particle images and projection images should be compared prior to searching for the next rotation and translation. Therefore, there is an import problem in the parameters search, as the GPU cannot afford to store the total number of particle images, which might be hundreds of GB in size. Hence, we changed the order of loop cycles as shown in Algorithm 1.

Algorithm 2 requires three parameters: sets of rotation angles, in-plane translations, particle images and 3D model, also called 3D volume. Among these three sets, R and T are special. We assumed that $R = \{R_1, \ldots, R_n\}$, where n is the total number of rotation angles. Each element in R such as R_i is a quaternion composed of four numbers that represent the particle rotation angles. Furthermore, we assumed that $T = \{T_1, \ldots, T_n\}$, where n is the total number of in-plane translations. Each element in T such as T_i is a key-value pair of the two numbers such as $< x, y >$, representing the displacement along the x-axis and y-axis, respectively. As shown in Algorithm 2, the image for-loop is the outermost loops. Hence, we use data blocks to complete the data transmission. In addition, Algorithm 2 showed that all calculations can be performed independently inside a for-loop, for example, between the (i-1)-th loop and the i-th loop inside translations

loop presented in the tenth line of Algorithm 2. Hence, we combined some loops as one GPU kernel to improve the throughput. Most of the GPU has three engines, including two copy and one computing engine. Hence, three streams are used in our implementation to overlap the transmission time and calculation time.

Figure 2 shows how we used the three streams to overlap transmission time and used the default stream for implicit synchronization. Furthermore, in all calculations, the computation is independent of different particle images, even between pixels in the same particle image. So, pixel level parallelism can also be termed fine-grained parallelism implemented in kernels, which enables better speed-up in our programs. As shown in Figure 1, in the M-step, the input particle images have to be inserted into a 3D volume at all possible rotation angles and translations provided by the previous step. In contrast to the E-step of CPU version, this approach of inserting images can easily divide images into blocks. Then the stream approach can be applied to overlap transmission and calculation time.

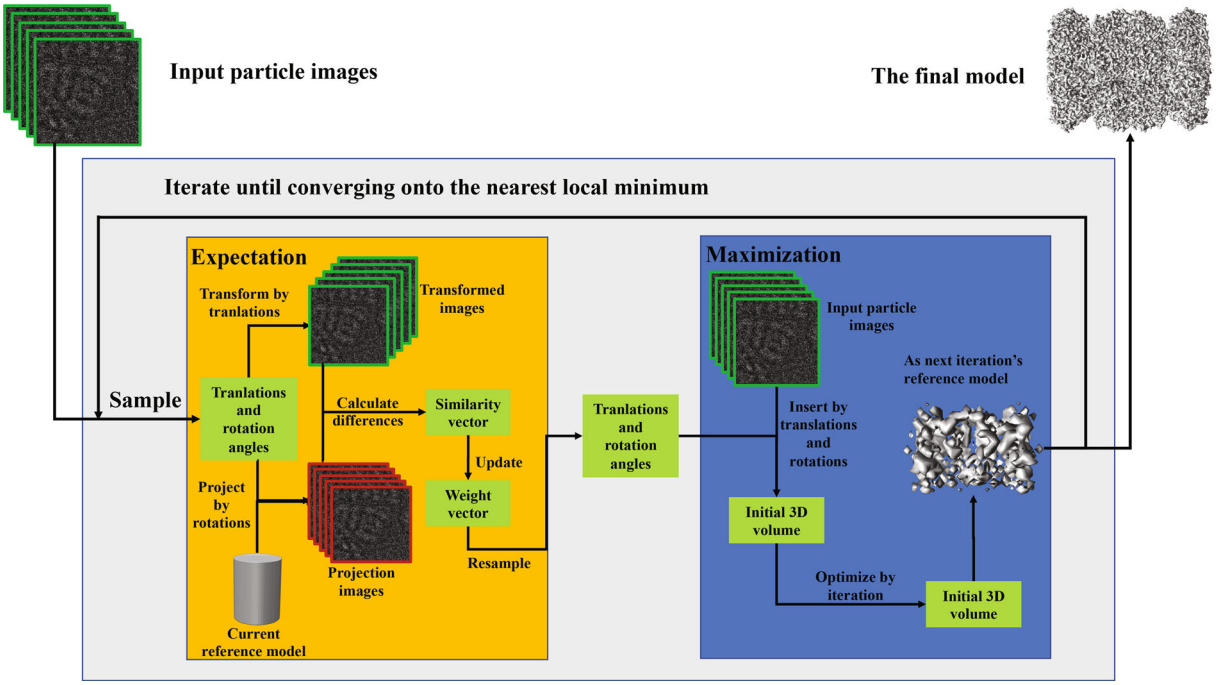

FIGURE 1 The 3D reconstruction algorithm workflow. In the E-step, the input is all particle images and the latest 3D volume reconstructed from the previous iteration. The output is the possible alignment parameters such as rotation angles and translations of each input image in the 3D volume. In the M-step, the input is the possible rotation angles and translations of each input image newly calculated in the E-step, and the output is the resulting 3D volume of the current iteration. During the execution of the algorithm, repeat the above expectation and M-step until the limit resolution of the 3D volume ceases to increase.

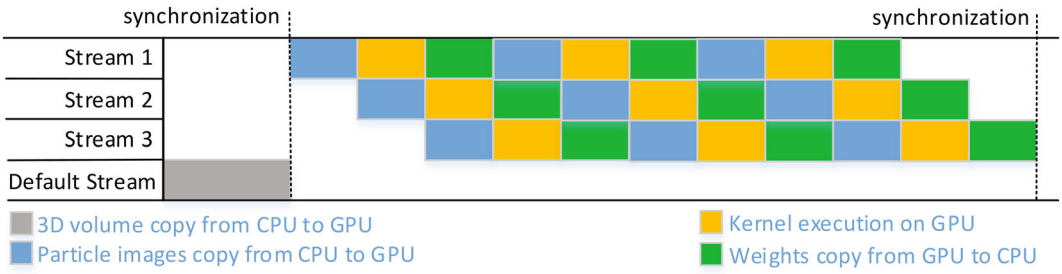

FIGURE 2 How data transmission and kernel calculations overlap. Operations are indicated by different colors. Yellow rectangles indicate a series of calculation kernels and not simply one calculation. The first synchronization is caused by the default stream's implicit synchronization;[22] otherwise, it is performed by manual addition.

ALGORITHM 2. Framework of parameter search in the E-step for GPU

Input: Rotation angles (R), Translations (T), Images (I), Volume (V).
Output: The probability density weight vector of R and T.

Initializing R and T
for $t \leftarrow 0$ to $NT - 1$ **do**
    extract in-plane translation T_t from T
    use T_t to calculate the translation distance called $TRAP_t$
end for
copy $TRAP$ data to GPU global memory
for $r \leftarrow 0$ to $NR - 1$ **do**
    extract out of plane rotation R_r from R
    calculate projection called $ROTP_r$ through R_r and V
end for
for $batch \leftarrow 0$ to $(NI - 1) * (NR - 1)$ **do**
    copy rotation data $ROTP_{batch}$ asynchronously from CPU to GPU global memory
    copy image data I_{batch} asynchronously from CPU to GPU global memory
    for $t \leftarrow 0$ to $NT - 1$ **do**
        multiply $ROTP_{batch}$ and $TRAP_{batch,t}$ to derive current spatial parameter variable called $RT_{batch,t}$
        calculate the difference $DIFF_{batch,t}$ through I_{batch} and $RTbatch, t$
        use $DIFF_{batch,t}$ to update the orientation weight W_r and W_t
    end for
    copy the orientation weight W_r and W_t data asynchronously from GPU global memory to CPU
end for
resample R and T

2.2 | Memory optimization

There are two types of storage component on the GPU device, namely, off-chip and on-chip storage components. Off-chip storage components include global memory, which contains large storage space albeit with high memory access latency. Mainstream GPU devices contain more than 10 GB storage space, where the access latency is considerably high, often reaching up to hundreds of CPU cycles. On-chip storage components include shared memory, constant memory, and texture cache.[23] On-chip memory usually has extremely low access latency but with limited storage space. For example, the current high-performance Nvidia P100 GPU card contains only 5 MB shared memory, 64 KB constant memory, and 128 KB texture cache. Although the size of on-chip storage is limited, its access latency is extremely low, and it usually requires only a few clock cycles to read and write data, indicating that the bandwidth of on-chip memory is at least ten times higher than that of off-chip memory.

Therefore, to maximize the performance of the GPU application, the most basic approach is to utilize on-chip memory maximally. For instance, data blocks that require frequent reading and writing should be put in shared memory as much as possible, whereas data blocks with unchanged values should be put in constant memory. In addition, use of texture memory for data storage is a good idea if the data blocks accessed by the program exhibit good spatial locality.

The 3D reconstruction algorithm involves many computational processes that require manipulation of 3D volumes, such as 3D volume projection operations in the E-step and weighted back projection in the M-step. In these processes, we often require logical coordinates of voxels in the 3D volume for the calculation. In memory, voxels are stored in a one-dimensional array, which is stored continuously in the direction of the x-axis. For example, we assumed that V is one of our 3D cube models, that is, it is a set of voxels such that: $V = \{v_0, v_1, \ldots, v_{n-1}\}$, where n is the number of voxels, with side length m. Next, we assumed that the center of the 3D coordinate axis overlaps with the lower left corner of the cube. Then, v_1 represents the point $(0, 0, 0)$, and v_2 represents the point $(1, 0, 0)$. Hence, we cannot access point $(0, 0, 0)$ and point $(0, 1, 0)$ in memory at the same time as the index of point $(0, 0, 0)$ in memory array is 0, whereas the index of point $(0, 1, 0)$ is

$$index = 0 * m * m + 1 * m + 0 = m. \quad (1)$$

```
//iCol: the coordinate of X-axis, iRow: the coordinate of Y-axis
//iSlc: the coordinate of Z-axis, dim: the length of a cube
for (int k = 0; k < 2; k++)
  for (int j = 0; j < 2; j++)
    for (int i = 0; i < 2; i++)
    {
      // get value by CPU memory
      index=(iSlc+k)*dim*dim+(iRow+j)*dim+(iCol+i);
      complex cval=value[index]

      // get value by GPU texture memory
      float2 cvalF=tex3D<float2>(value,iCol+i,iRow+j,iSlc+k);
    }
```

FIGURE 3 Usage of GPU texture memory.

Therefore, when these processes are required, we transfer the volume data from CPU memory to GPU texture memory instead of using the GPU global memory. As shown in Figure 3, GPU texture memory allows us to access data directly from 3D coordinates and achieves small access latency.

Furthermore, prior calculation of the rotation angle, followed by rotation of the image according to the angle is always required both in expectation and M-step. Meanwhile, pixels in the same image need to rotate by the same angle; hence, we use shared memory, which is readable and writable, to store these rotations in order to speed up the access latency. Furthermore, in the M-step, weighted back projection algorithm requires particle images inserted in 3D volume with weights. These weights will not be changed during the insertion operations and pixels in the same image will share the same weights. Thus, constant memory is the best choice for storing these weights.

2.3 | Precision optimization

Currently, mainstream GPU devices generally support arithmetic operations for three precision types of floating-point numbers, namely half-precision, single-precision, and double-precision. Theoretically, in terms of computational speed, half-precision floating-point numbers are two times faster than single-precision floating-point numbers, and four times faster than the double-precision floating-point numbers. On storage overhead, half-precision floating-point numbers are one-half of the single-precision floating-point number and are one-quarter of the double-precision floating-point number. Therefore, the application program has to carefully select the optimal floating-point data type for relevant calculation according to its own calculation characteristics. Generally speaking, for applications where the demand for accuracy is not very high, use of a semi-precision or single-precision is better for arithmetic operations. For applications with high precision requirements, unless the calculation result is extremely sensitive to numerical precision, double precision should be used for calculation; otherwise, we can consider reducing the precision for calculation.

In the cryo-EM field, the original raw data collected is grey-scale maps, which is stored using integers in the memory. Thus, the required precision depends on the algorithms. Whether we can speed up computation and storage by reducing precision depends on the effect on the resolution quality of the results. However, the maximum representable value of the half-precision is 65,504, which is far from meeting our needs. Thus we have to choose from single-precision and double-precision. 3D volume rotation, projection onto a 2D image and back-projection of a 2D image into a 3D volume are common operations used in 3D reconstruction algorithm. In these operations, the rotation angles are decimals, and therefore, the coordinates after rotation are also decimals. In contrast, the pixel coordinates in the 2D image or the voxel coordinates in the 3D volume are integers. Thus, interpolation algorithms are required for these operations. For instance, when tri-linear interpolation is used to compute 3D volume projection, we assumed that the coordinates of a point in a 3D volume after rotation are (x_0, y_0, z_0). Therefore, this point corresponds to the pixel with a coordinate of $(\lfloor x_0 \rfloor, \lfloor y_0 \rfloor)$ in the projection image. According to the principle of tri-linear interpolation, one of the weights interpolated along the x-axis is

$$x_d = x_0 - \lfloor x_0 \rfloor . \tag{2}$$

and the other is $(1 - x_d)$. The weights of the y-axis and z-axis can be solved equally. We observed that x_d is actually the fractional part of x_0. However, the number of significant figures in single-precision floating-point numbers is 7 digits, and

the number of significant figures in double-precision is 15 digits. Therefore, in the case of using single-precision, if the integer part of the coordinate is large, such as 123456.789, there will be almost no significant digit in the decimal part. In this case, the results of the tri-linear interpolation will be wrong. To summarize, we have very high precision demand for the rotation angle value of the 2D image and 3D volume. Thus, we chose to use double-precision for calculating the rotations; in other cases, single-precision was used.

3 | RESULTS

All presented optimization (Section 2) were implemented. In the rest of this section, we will introduce all the experiments for testing its performance.

3.1 | Experimental setup

The evaluation was performed on a heterogeneous computing platform as shown in Table 1. This platform is a server cluster with 20 nodes. Each node has the same configuration: two Intel Xeon (Broadwell) CPUs, four NVIDIA K40 GPUs, and 256 GB DDR4 2400 MHz RAM with 32 KB L1 cache, 256 KB L2 cache, and a 20 MB shared L3 cache.

The CPUs are 12-core Intel Xeon E5-2643 v4 processor based on the Intel Broadwell microarchitecture running at 3.40 GHz, whereas the GPUs are 2880-core processor based on Kepler architecture with 12 GB global memory, 64 KB constant memory, and 48 KB shared memory per thread block. The GPUs in each node are connected with PCIE. The Operation System is 64-bit Red Hat Enterprise Linux Server release 7.4 (Maipo). The Broadwell architecture uses 256-bit AVX vector instruction sets. We used GCC v4.8.5 compilers, MPICH v3.3, and CUDA v10.2. In addition, we also used NVIDIA Collective Communications Library (NCCL) 2.1.15, a CUDA communication library, in our code for accelerating collective communication primitives.

To complete the performance experiments, we used three high-resolution datasets: the Thermoplasma acidophilum 20S proteasome[24] (Entry code: 10025) and TRPV1[25] (Entry code: 10059) from the electron microscopy public image archive (EMPIAR), and the cyclic-nucleotide-gated[26] (CNG) dataset from our previous work. We can identify these density maps from the electron microscopy data bank (EMDB) with their entry codes 6287, 8117, and 6656. Furthermore, the entry codes of the structure models from Protein Data Bank (PDB) are 1PMA, 5irx, and 5H3O. The particle box size of an image, the particle image pixel size, and total particle image number of three datasets are shown in Table 2.

Based on their structural differences, different initial reference models are used for three datasets. For TRPV1 and CNG, we used the density maps of these proteins downloaded from EMDB. Nevertheless, a cylinder model was used as the initial reference model for the proteasome.

TABLE 1 The configurations of computing platform

Platform	CPU configurations	GPU configurations
Chip type	Intel Xeon E5-2643 v4	Nvidia K40
Memory	256 GB	12 GB
Operation system	64-bit Red Hat 7.4	
Softwares	GCC v4.8.5, MPICH v3.3, cuda v10.2, NCCL v2.1.15	

TABLE 2 Three protein datasets with different conformations

Datasets	Size of particle box	Pixelsize of particle image	Total particle image number
CNG	160×160	1.3200	211826
Trpv1	192×192	1.2156	151504
Proteasome	320×320	1.0520	131319

3.2 | Performance evaluation

Prior to all performance experiments, we determined whether gTHUNDER can generate the same final reconstruction quality as our benchmark THUNDER with single-precision.

We evaluated this primarily by examining the agreement of refinement results using density maps and the gold standard of fourier shell correlation (FSC) curves on three datasets. Figure 4 shows that gTHUNDER can achieve almost the same FSC curve as the benchmark. In addition, a combination of the representative secondary structures segmented from the maps deposited in EMDB and the density map calculated using gTHUNDER or the benchmark demonstrated that gTHUNDER is qualitatively identical to the benchmark. Two criteria are used to evaluate the performance of gTHUNDER, namely, speed-up ratio and scalability. Without special explanation, we are discussing the performance of the mixed-precision version of gTHUNDER.

3.2.1 | The performance of the speed-up ratio

We used the Thermoplasma acidophilum 20S proteasome dataset and 18 nodes in the cluster to compare the speed-up in each iteration round during the running of the program. Both the benchmark and gTHUNDER have been calculated in 35 iterations, eventually reaching a resolution of 2.45Å. The benchmark takes a total of 907 min and the gTHUNDER takes a total of 145 min, so the total running time achieved a 6.26 × speed-up.

As shown in Figure 5A, with the increase of iterations and resolution, the running time of one iteration increases gradually. In the meantime, the speed-up ratio also increases. From global search to local search and even to CTF search, the increase in resolution will lead to more and more calculations. And the increase in calculations will make our work achieve more speed-up ratio. In addition, from Figure 5B, we can find that when switching from global search to local search, or from local search to CTF search, a short-term decrease in the speed-up ratio occurs. This is because the switching of the search type will bring the calculations fluctuation. But in general, our work has achieved a huge increase in total running time from tens of hours to hours.

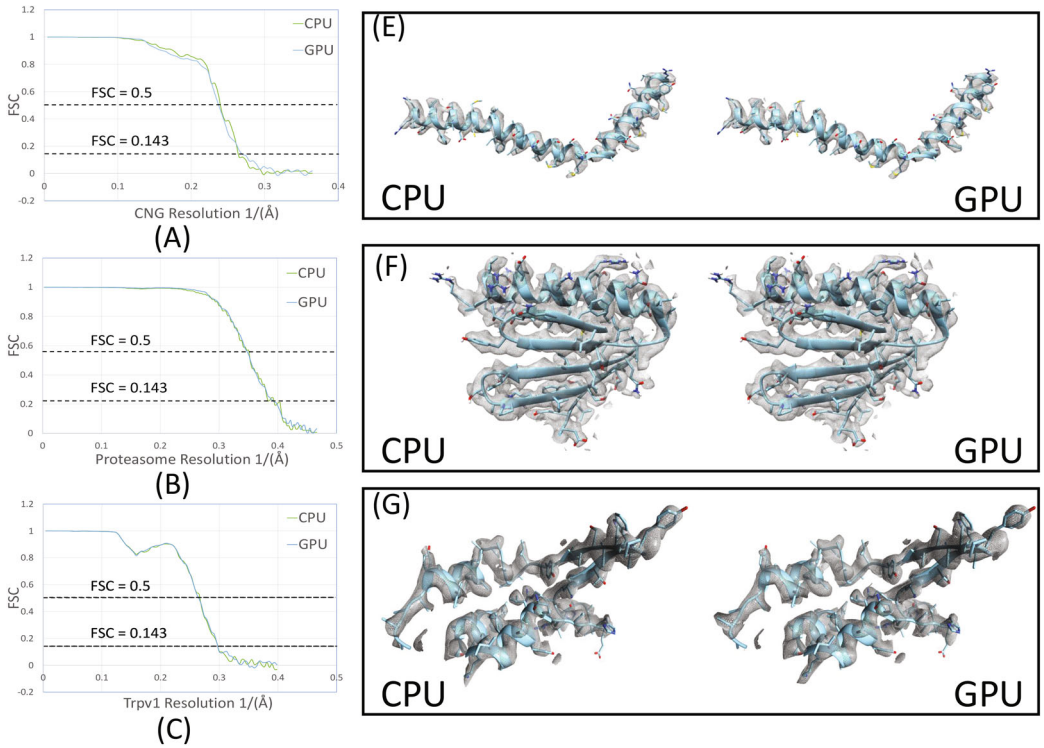

FIGURE 4 Comparative results of the final reconstruction quality between gTHUNDER and the benchmark CPU version. (A) to (C) are the FSC curve diagram of final resolution results. (E) to (G) are the secondary structures and the density maps.

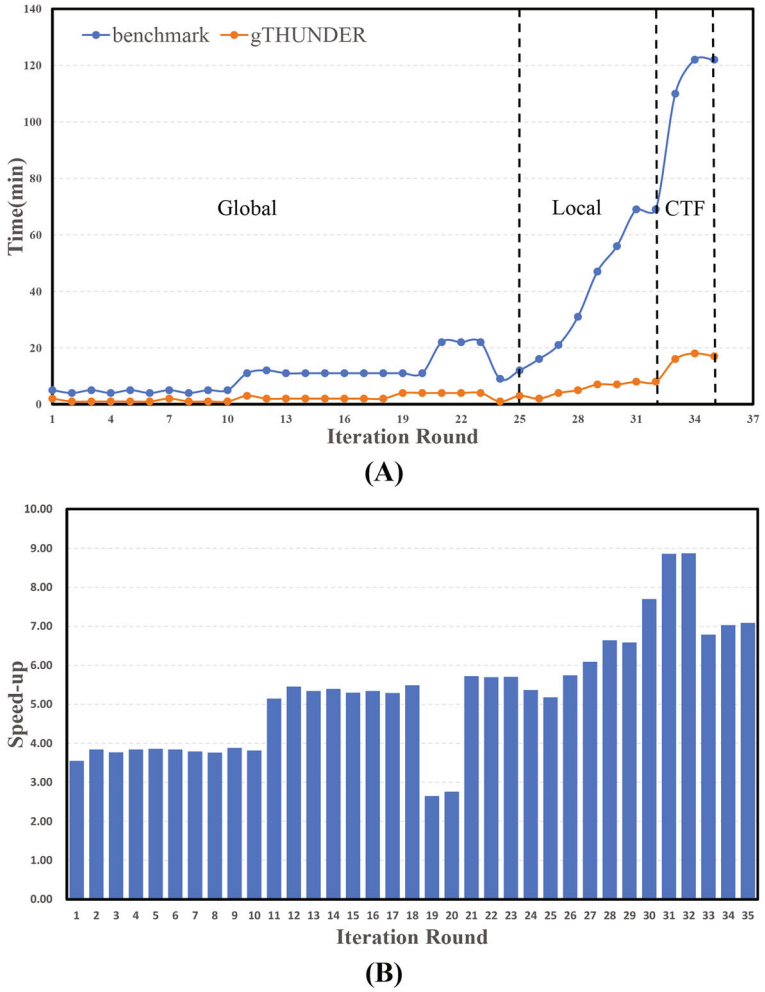

FIGURE 5 Comparison of the program running time of the Thermoplasma acidophilum 20S proteasome dataset. (A) shows the program running time of each iteration round. There are 25 rounds of global search, 7 rounds of local search and 3 rounds of CTF search. (B) shows the speed-up of each iteration round.

3.2.2 | The effect of precision on the speed-up ratio

We used two different numbers of nodes (6 and 16) in the cluster to compare the speed-up caused by precision differences between the benchmark and our work. As shown in Table 3, the speed-up of different datasets on mixed-precision and double-precision are almost identical (nearly two times). This trend was expected, as the number of computations, storages, and communications were halved from double-precision to single-precision. Table 3 suggest that the speed-up of the three datasets varies within an interval. This is because computations and communications increase at different rates with the increase of input data. The speed-up result of the three datasets indicates that increasing the size of the particle images and the number of particle images will generate better speed-up.

TABLE 3 Compared total execution time (in [s]) between benchmark, gTHUNDER mixed-precision, and double-precision

	Benchmark version		GPU-double version		GPU-mixed version	
Datasets	**6**	**16**	**6**	**16**	**6**	**16**
CNG	72,576	26,280	19,884	7394	11,520	4352
Trpv1	42,554	17,408	12,055	4975	6784	3328
Proteasome	186,586	59,700	47,842	17,833	27,520	9728

TABLE 4 Execution times (in [s]) with our methods on the GPU cluster

DataSets \ Nodes	2	4	6	8	10	12	14	16
CNG	32,640	19,072	11,520	8704	6720	6144	4992	4352
Trpv1	19,456	10,240	6784	5376	4224	3840	3584	3328
Proteasome	63,616	33,152	27,520	17,792	15,360	13,560	10,880	9728

Furthermore, Table 3 shows that compared to the benchmark implementation, gTHUNDER in mixed-precision achieved 5-7-fold performance improvement under the same number of cluster nodes. Based on our purchase price, one cluster node without GPU requires approximately fifty thousand RMB, whereas one GPU of NVIDIA TESLA K40 requires approximately twenty-five thousand RMB. Thus, three cluster nodes without GPU have almost the same hardware costs as one cluster node with four NVIDIA TESLA K40. Considering the same hardware costs, Tables 3 and 4 show that gTHUNDER can be nearly three times faster than the benchmark.

3.2.3 | Performance evaluation of scalability

As for scalability, our experimental scheme is from two cluster nodes to 16 cluster nodes, with two as the step size. Table 4 shows the absolute execution times (in [s]) on the cluster between the different numbers of cluster nodes. Table 4 also shows that for data of fixed size, the time required for calculation decreased with the number of cluster nodes. We calculated the speed-up ratio and the parallel efficiency of other number of nodes based on the total running time of the two nodes in Table 4. Figure 6A shows that gTHUNDER achieves a near-linear speed-up on the cluster, which indicates that gTHUNDER has a considerablely strong scalability. To calculate the parallel efficiency, we consider that N_i represents

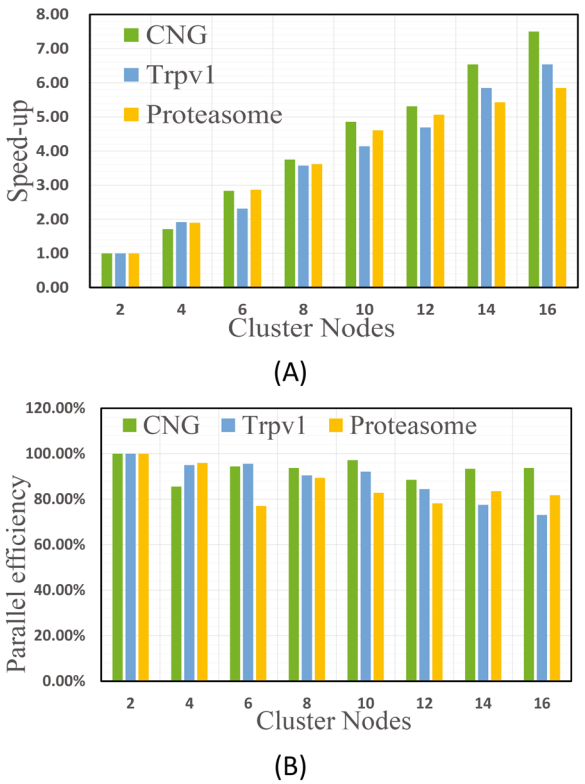

FIGURE 6 Strong scalability on the cluster with GPUs for three datasets. Based on the program running time of the two nodes, (A) shows the speed-up of the different number of cluster nodes, (B) shows the parallel efficiency of the different number of cluster nodes.

that there are i nodes in the cluster and T_{N_i} represents the total running time when the cluster has i nodes. Then, the parallel efficiency (PE) of the program can be calculated in the following way:

$$PE = \frac{(T_{N_i} - T_{N_j}) \times N_i}{T_{N_i} \times N_j}. \tag{3}$$

As shown in Figure 6B, gTHUNDER achieves high parallel efficiency with an average of over 84%. Our high parallel efficiency means that when using large-scale clusters for computing, we can also take full advantage of all the computing resources of the cluster. However, strong scalability is affected by both parallelizable and non-parallelizable parts, such as particle image data reading, and communication parts. Despite our program's excellent strong scalability, there will still be an upper limit of the speed-up according to Amdahl's law as the cluster nodes continue to expand. In addition, according to the experiment results, the final reconstruction resolution quality is not influenced by the increase in the number of nodes, which ensures the robustness of our work.

4 | CONCLUSION

In this paper, we proposed a series of approaches for transplanting the CPU program to GPU to achieve high speed-up and extraordinary strong scalability. These approaches need to be compatible with the GPU hardware features from the computational logical level of the 3D reconstruction algorithm. These adjustments are universal in the field of cryo-EM 3D reconstruction and can be applied to other programs based on Bayesian-likelihood approaches in structural biology. In the performance experiments, we used three datasets with different characteristics to evaluate the performance compared to the benchmark program. Experiments show that considering the same hardware costs, gTHUNDER can be nearly three times faster than the benchmark. Furthermore, the average parallel efficiency can reach up to 84%. The protein structure that was previously unable to calculate due to insufficient computational time and computational resources now can be calculated. These approaches can be widely applied to high-performance computing in the field of structural biology.

AUTHOR CONTRIBUTIONS

Zhao Wang: conceptualization (lead); data curation (lead); formal analysis (lead); funding acquisition (lead); investigation (lead); methodology (lead); software (lead); validation (equal); visualization (equal); writing – original draft (lead); writing – review and editing (equal). **Huabin Ruan:** conceptualization (supporting); data curation (supporting); formal analysis (supporting); funding acquisition (supporting); investigation (supporting); methodology (supporting); software (supporting); validation (equal); visualization (equal); writing – original draft (supporting); writing – review and editing (equal). **Guangwen Yang:** project administration (equal); resources (equal); software (supporting); supervision (equal); validation (supporting); visualization (supporting); writing – review and editing (supporting). **Xueming Li:** data curation (supporting); project administration (equal); resources (equal); software (supporting); supervision (equal); validation (supporting); visualization (supporting); writing – review and editing (supporting).

ACKNOWLEDGMENTS

This work was supported by funds from Advanced Innovation Center for Structural Biology (to Xueming Li and Guangwen Yang), Tsinghua-Peking Joint Center for Life Sciences (to Xueming Li), One-Thousand Talent Program through the State Council of China (to Xueming Li), and Intel Parallel Computing Center project (to Xueming Li).

CONFLICT OF INTEREST

Authors have no conflict of interest relevant to this article.

PEER REVIEW

The peer review history for this article is available at https://publons.com/publon/10.1002/eng2.12601.

DATA AVAILABILITY STATEMENT

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

ORCID

Zhao Wang https://orcid.org/0000-0001-7319-7615

REFERENCES

1. Cheng Y. Single-particle cryo-em at crystallographic resolution. *Cell*. 2015;161(3):450-457.
2. Lyumkis D, Brilot AF, Theobald DL, Grigorieff N. Likelihood-based classification of cryo-em images using frealign. *J Struct Biol*. 2013;183(3):377-388.
3. Scheres SH. Beam-induced motion correction for sub-megadalton cryo-em particles. *eLife*. 2014;3:e03665.
4. Rubinstein JL, Brubaker MA. Alignment of cryo-em movies of individual particles by optimization of image translations. *J Struct Biol*. 2015;192(2):188-195.
5. Scheres SH. Relion: Implementation of a Bayesian approach to cryo-em structure determination. *J Struct Biol*. 2012;180(3):519-530.
6. Punjani A, Rubinstein JL, Fleet DJ, Brubaker MA. cryosparc: algorithms for rapid unsupervised cryo-em structure determination. *Nat Methods*. 2017;14:290-296.
7. Hu M, Yu H, Gu K, et al. A particle-filter framework for robust cryo-em 3d reconstruction. *Nat Methods*. 2018;15(12):1083-1089.
8. Elmlund D, Elmlund H. Simple: software for ab initio reconstruction of heterogeneous single-particles. *J Struct Biol*. 2012;180(3):420-427.
9. Arulampalam MS, Maskell S, Gordon N, Clapp T. A tutorial on particle filters for online nonlinear/non-gaussian Bayesian tracking. *IEEE Trans Signal Process*. 2002;50(2):174-188.
10. Gustafsson F, Gunnarsson F, Bergman N, et al. Particle filters for positioning, navigation, and tracking. *IEEE Trans Signal Process*. 2002;50(2):425-437.
11. Kimanius D, Forsberg BO, Scheres SH, Lindahl E. Accelerated cryo-em structure determination with parallelisation using gpus in relion-2. *eLife*. 2016;5:e18722.
12. Scheres SH. Chapter eleven - classification of structural heterogeneity by maximum-likelihood methods. *Cryo-EM, Part B: 3-D Reconstruction*. Vol 482. Academic Press; 2010:295-293.
13. Yin Z, Zheng Y, Doerschuk PC, Natarajan P, Johnson JE. A statistical approach to computer processing of cryo-electron microscope images: virion classification and 3-d reconstruction. *J Struct Biol*. 2003;144(1):24-50.
14. Scheres SH. A Bayesian view on cryo-em structure determination. *J Mol Biol*. 2012;415(2):406-418.
15. Sigworth F. A maximum-likelihood approach to single-particle image refinement. *J Struct Biol*. 1998;122(3):328-339.
16. Abraham MJ, Murtola T, Schulz R, et al. GROMACS: high performance molecular simulations through multi-level parallelism from laptops to supercomputers. *SoftwareX*. 2015;1-2:19-25.
17. Dempster A, Laird N, Rubin D. Maximum likelihood from incomplete data via em algorithm. *J. Royal Statistical Soc., Series B*. 1997;39:1-38.
18. Scheres SH, Valle M, Carazo JM. Fast maximum-likelihood refinement of electron microscopy images. *Bioinformatics*. 2005;21(2):243-244.
19. Tagare HD, Barthel A, Sigworth FJ. An adaptive expectation–maximization algorithm with gpu implementation for electron cryomicroscopy. *J Struct Biol*. 2010;171(3):256-265.
20. Díez DC, Mueller H, Frangakis AS. Implementation and performance evaluation of reconstruction algorithms on graphics processors. *J Struct Biol*. 2007;157(1):288-295.
21. Castaño-Díez D, Moser D, Schoenegger A, Pruggnaller S, Frangakis AS. Performance evaluation of image processing algorithms on the gpu. *J Struct Biol*. 2008;164(1):153-160.
22. Buck I, Foley T, Horn D, et al. Brook for gpus: Stream computing on graphics hardware. *ACM Trans. Graph*. 2004;23(3):777-786.
23. Hoang TV, Cavin X, Ritchie D. Gemfitter: A highly parallel fft-based 3d density fitting tool with gpu texture memory acceleration. *J Struct Biol*. 2013;184(2):348-354.
24. Campbell MG, Veesler D, Cheng A, Potter CS, Carragher B. 2.8Å resolution reconstruction of the thermoplasma acidophilum 20s proteasome using cryo-electron microscopy. *eLife*. 2015;4:e06380.
25. Gao Y, Cao E, Julius D, Cheng Y. Trpv1 structures in nanodiscs reveal mechanisms of ligand and lipid action. *Nature*. 2016;534:347-351.
26. Li M, Zhou X, Wang S, et al. Structure of a eukaryotic cyclic-nucleotide-gated channel. *Nature*. 2017;542:60-65.

How to cite this article: Wang Z, Ruan H, Yang G, Li X. Parallelizing the cryo-EM structure determination in THUNDER using GPU cluster. *Engineering Reports*. 2025;7(1):e12601. doi: 10.1002/eng2.12601